

Evaluation Module

The evaluation module provides comprehensive tools for assessing blood glucose forecasting models. It includes statistical metrics, clinical error grid analysis, visualization tools, and reporting utilities.

Table of Contents

- [Overview](#)
- [Module Structure](#)
- [Core Components](#)
- [Usage Examples](#)
- [Metrics Reference](#)
- [Visualization Guide](#)
- [Reporting Tools](#)
- [Installation](#)

Overview

This module implements industry-standard evaluation methods for blood glucose prediction systems, including:

- **Statistical Metrics:** RMSE, MAE, MAPE, R^2 , correlation
- **Clinical Error Grids:** Clarke and Parkes Error Grid Analysis
- **Visualizations:** Prediction plots, comprehensive dashboards
- **Reporting:** Export to CSV, JSON, LaTeX, and Markdown formats

Module Structure

```
evaluation/
├── __init__.py          # Module initialization and exports
├── metrics.py           # Statistical metrics and BGMetrics class
├── visualisation.py     # Plotting functions and dashboards
├── reporting.py         # Report generation and export utilities
├── clarke_ega/          # Clarke Error Grid Analysis
│   ├── __init__.py
│   ├── clarke_ega.py    # ClarkeEGA implementation
│   └── README.md        # Clarke EGA documentation
├── parkes_ega/          # Parkes Error Grid Analysis
│   ├── __init__.py
│   └── boundaries.py    # Boundary definitions for Parkes
└── implementation
    ├── parkes_ega.py    # ParkesEGA implementation
    └── README.md        # Parkes EGA documentation
```

Core Components

1. Metrics ([metrics.py](#))

The [BGMetrics](#) class provides comprehensive metric calculation:

```
from benchmark.evaluation.metrics import BGMetrics
import numpy as np

# Your data
y_true = np.array([120, 135, 142, 128, 115])
y_pred = np.array([118, 138, 140, 130, 117])

# Calculate all metrics at once
metrics = BGMetrics.calculate_comprehensive_metrics(y_true, y_pred)

print(f"RMSE: {metrics['rmse']:.2f} mg/dL")
print(f"MAE: {metrics['mae']:.2f} mg/dL")
print(f"Clarke Zone A: {metrics['clarke_zones']['A']:.1f}%")
print(f"Parkes Zone A: {metrics['parkes_zones']['A']:.1f}%")
```

Available Metrics:

- [rmse](#): Root Mean Squared Error
- [mae](#): Mean Absolute Error
- [mape](#): Mean Absolute Percentage Error
- [mard](#): Mean Absolute Relative Difference
- [time in range correlation](#): Percentage of readings within target range
- [clarke_zones](#): Dictionary of Clarke EGA zone percentages (A-E)
- [parkes_zones](#): Dictionary of Parkes EGA zone percentages (A-E)

2. Clarke Error Grid Analysis ([clarke_ega/](#))

Clinical accuracy assessment based on Clarke's 1987 methodology. See [clarke_ega/README.md](#) for detailed documentation.

```
from benchmark.evaluation.clarke_ega import ClarkeEGA

clarke = ClarkeEGA()
results = clarke.analyze(y_true, y_pred)

print(f"Zone A (Acceptable): {results['percentages']['A']:.1f}%")
print(f"Zone B (Benign): {results['percentages']['B']:.1f}%")
```

Clinical Interpretation:

- **Zone A**: Clinically accurate values (>95% recommended)
- **Zone B**: Benign errors with no treatment effect
- **Zone C**: Overcorrection errors

- **Zone D:** Dangerous failure to detect hypoglycemia
- **Zone E:** Erroneous treatment decisions

3. Parkes Error Grid Analysis ([parkes_ega/](#))

Consensus Error Grid for Type 1 diabetes. See [parkes_ega/README.md](#) for detailed documentation.

```
from benchmark.evaluation.parkes_ega import ParkesEGA

parkes = ParkesEGA()
results = parkes.analyze(y_true, y_pred)

print(f"Zone A (Acceptable): {results['percentages']['A']:.1f}%")
```

4. Visualization ([visualisation.py](#))

Plotting functions for model evaluation:

```
from benchmark.evaluation.visualisation import (
    plot_predictions,
    plot_clarke_analysis,
    plot_parkes_analysis,
    create_prediction_dashboard
)

# Create comprehensive dashboard
fig = create_prediction_dashboard(
    y_true=y_true,
    y_pred=y_pred,
    metrics=metrics,
    title="LSTM Model Evaluation",
    save_path="dashboard.png"
)
```

Available Plots:

- [plot_predictions\(\)](#): Time series comparison with confidence intervals
- [plot_clarke_analysis\(\)](#): Clarke Error Grid with zone annotations
- [plot_parkes_analysis\(\)](#): Parkes Error Grid with zone annotations
- [create_prediction_dashboard\(\)](#): Comprehensive 6-panel dashboard with the complete plotting

5. Reporting ([reporting.py](#))

Export and report generation utilities:

```

from benchmark.evaluation.reporting import (
    export_metrics_to_csv,
    export_metrics_to_json,
    generate_latex_report,
    generate_model_comparison_report
)

# Export metrics to CSV (ordered: statistics → Clarke → Parkes)
export_metrics_to_csv(
    metrics=metrics,
    filepath="results/model_metrics.csv",
    model_name="LSTM"
)

# Export to JSON (ordered structure)
export_metrics_to_json(
    metrics=metrics,
    filepath="results/model_metrics.json",
    model_name="LSTM",
    patient_id="patient_540"
)

# Generate LaTeX report with visualizations
latex_path = generate_latex_report(
    model_metrics={'LSTM': lstm_metrics, 'GRU': gru_metrics},
    y_true_dict={'LSTM': y_true_lstm, 'GRU': y_true_gru},
    y_pred_dict={'LSTM': y_pred_lstm, 'GRU': y_pred_gru},
    output_dir="reports/",
    report_title="Blood Glucose Forecasting Model Comparison"
)

```

Usage Examples

Basic Evaluation

```

import numpy as np
from benchmark.evaluation import BGMetrics, create_prediction_dashboard

# Generate or load your predictions
y_true = np.array([...]) # True glucose values
y_pred = np.array([...]) # Predicted glucose values

# Calculate metrics
metrics = BGMetrics.calculate_comprehensive_metrics(y_true, y_pred)

# Create visualization
fig = create_prediction_dashboard(
    y_true=y_true,
    y_pred=y_pred,

```

```

        metrics=metrics,
        save_path="evaluation_dashboard.png"
    )

    print(f"Model Performance:")
    print(f"    RMSE: {metrics['rmse']:.2f} mg/dL")
    print(f"    Clarke Zone A: {metrics['clarke_zones']['A']:.1f}%")
    print(f"    Parkes Zone A: {metrics['parkes_zones']['A']:.1f}%")

```

Multi-Model Comparison

```

from benchmark.evaluation.reporting import (
    generate_model_comparison_report,
    generate_latex_report
)

# Evaluate multiple models
models = {
    'LSTM': (y_true_lstm, y_pred_lstm),
    'GRU': (y_true_gru, y_pred_gru),
    'Transformer': (y_true_transformer, y_pred_transformer)
}

model_metrics = {}
y_true_dict = {}
y_pred_dict = {}

for model_name, (y_true, y_pred) in models.items():
    model_metrics[model_name] =
    BGMetrics.calculate_comprehensive_metrics(y_true, y_pred)
    y_true_dict[model_name] = y_true
    y_pred_dict[model_name] = y_pred

# Generate comparison table
comparison_df = generate_model_comparison_report(
    model_metrics=model_metrics,
    output_format='dataframe'
)
print(comparison_df)

# Generate comprehensive LaTeX report
latex_path = generate_latex_report(
    model_metrics=model_metrics,
    y_true_dict=y_true_dict,
    y_pred_dict=y_pred_dict,
    output_dir="reports/",
    include_visualizations=True
)
print(f"Report generated: {latex_path}")

```

Patient-Specific Analysis

```
from benchmark.evaluation.reporting import (
    export_metrics_to_json,
    aggregate_patient_metrics
)

# Evaluate across multiple patients
patient_results = {}

for patient_id in ['patient_540', 'patient_544', 'patient_567']:
    y_true = load_patient_data(patient_id, 'true')
    y_pred = load_patient_data(patient_id, 'pred')

    metrics = BGMetrics.calculate_comprehensive_metrics(y_true, y_pred)
    patient_results[patient_id] = metrics

# Export individual patient results
export_metrics_to_json(
    metrics=metrics,
    filepath=f"results/{patient_id}_metrics.json",
    patient_id=patient_id
)

# Aggregate across patients
aggregated = aggregate_patient_metrics(patient_results)
print(f"Average RMSE: {aggregated['rmse']:.2f} mg/dL")
print(f"Average Clarke Zone A: {aggregated['clarke_zones']['A']:.1f}%")
```

Metrics Reference

Statistical Metrics

Metric	Formula	Interpretation	Acceptable Range
RMSE	$\sqrt{(\sum (y_{\text{true}} - y_{\text{pred}})^2 / n)}$	Lower is better	< 15 mg/dL (excellent) < 20 mg/dL (good)
MAE	$\sum y_{\text{true}} - y_{\text{pred}} / n$	Lower is better	< 10 mg/dL (excellent) < 15 mg/dL (good)
MAPE	$100 \times \sum y_{\text{true}} - y_{\text{pred}} / y_{\text{true}} / n$	Lower is better	< 10% (excellent) < 15% (good)
MARD	$\frac{1}{N} \sum_{k=1}^N 100\% \cdot \frac{ y_{\text{CGM}}(t_k) - y_{\text{ref}}(t_k) }{y_{\text{ref}}(t_k)}$		

Clinical Metrics (Error Grids)

Both Clarke and Parkes Error Grids categorize predictions into zones:

- **Zone A:** Clinically accurate (target: >95%)
- **Zone B:** Benign errors (acceptable: <5%)
- **Zone C:** Overcorrection errors (target: 0%)
- **Zone D:** Dangerous errors (target: 0%)
- **Zone E:** Erroneous treatment (target: 0%)

Clinical Acceptance Criteria:

- Zone A + B should be $\geq 99\%$
- Zone C + D + E should be $< 1\%$
- Ideally, Zone A should be $> 95\%$

Visualization Guide

Dashboard Components

The `create_prediction_dashboard()` function generates a comprehensive 6-panel visualization:

1. **Time Series Plot:** True vs. predicted glucose over time
2. **Metrics Summary:** Key performance indicators
3. **Clarke Error Grid:** Clinical accuracy assessment
4. **Parkes Error Grid:** Consensus error grid analysis

Reporting Tools

Export Formats

The module supports multiple export formats with consistent metric ordering:

Ordering Convention:

1. Metadata (model name, patient ID, timestamp)
2. Statistical metrics (RMSE, MAE, MAPE, etc.)
3. Clarke EGA zones (A, B, C, D, E)
4. Parkes EGA zones (A, B, C, D, E)

LaTeX Report Generation

Generate publication-ready LaTeX reports:

```
latex_path = generate_latex_report(  
    model_metrics=model_metrics,  
    y_true_dict=y_true_dict,  
    y_pred_dict=y_pred_dict,  
    output_dir="reports/",  
    report_title="My Research Report",  
    author="Research Team",  
    include_visualizations=True
```

```
)  
  
# Compile to PDF (requires LaTeX installation)  
import subprocess  
subprocess.run(['pdflatex', latex_path], cwd="reports/")
```

The generated report includes:

- Executive summary
- Statistical comparison tables
- Clarke and Parkes EGA tables
- Comprehensive visualizations for each model
- Conclusions and recommendations

Installation

Required Dependencies

```
pip install numpy pandas matplotlib scipy
```

Optional Dependencies

For full functionality:

```
pip install seaborn scikit-learn
```

LaTeX Support

For PDF report generation:

```
# Ubuntu/Debian  
sudo apt-get install texlive-full  
  
# macOS  
brew install mactex  
  
# Windows  
# Download and install MiKTeX from https://miktex.org/
```

API Reference

BGMetrics Class

```

class BGMetrics:
    @staticmethod
    def calculate_rmse(y_true, y_pred) -> float

    @staticmethod
    def calculate_mae(y_true, y_pred) -> float

    @staticmethod
    def calculate_mape(y_true, y_pred) -> float

    @staticmethod
    def calculate_comprehensive_metrics(y_true, y_pred) -> Dict

```

ClarkeEGA Class

See [clarke_ega/README.md](#) for complete documentation.

ParkesEGA Class

See [parkes_ega/README.md](#) for complete documentation.

Clinical Guidelines

FDA Recommendations

For regulatory approval of continuous glucose monitoring systems:

- 99% of values should be in Clarke Zones A+B
- 95% should be in Zone A
- 0% in Zones D+E

ISO 15197:2013 Standards

For blood glucose monitoring systems:

- 95% of results should be within ± 15 mg/dL of reference for glucose < 100 mg/dL
- 95% of results should be within $\pm 15\%$ of reference for glucose ≥ 100 mg/dL

Contributing

When adding new metrics or visualizations:

1. Follow the existing code structure
2. Add comprehensive docstrings
3. Include unit tests
4. Update this README
5. Ensure compatibility with the metric ordering convention

References

1. Clarke, W. L., et al. (1987). "Evaluating clinical accuracy of systems for self-monitoring of blood glucose." *Diabetes Care*, 10(5), 622-628.
2. Parkes, J. L., et al. (2000). "A new consensus error grid to evaluate the clinical significance of inaccuracies in the measurement of blood glucose." *Diabetes Care*, 23(8), 1143-1148.
3. ISO 15197:2013. "In vitro diagnostic test systems — Requirements for blood-glucose monitoring systems for self-testing in managing diabetes mellitus."

License

This module is part of the BG-Forecasting project. See the main LICENSE file for details.

Contact

For questions or issues, please open an issue on the GitHub repository.